GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

# Unitat 04 — Estructures de control i excepcions bàsiques
# Bloc 01

## 1r CFGS DAW · Programació

Sergi García Barea

# Control del flux

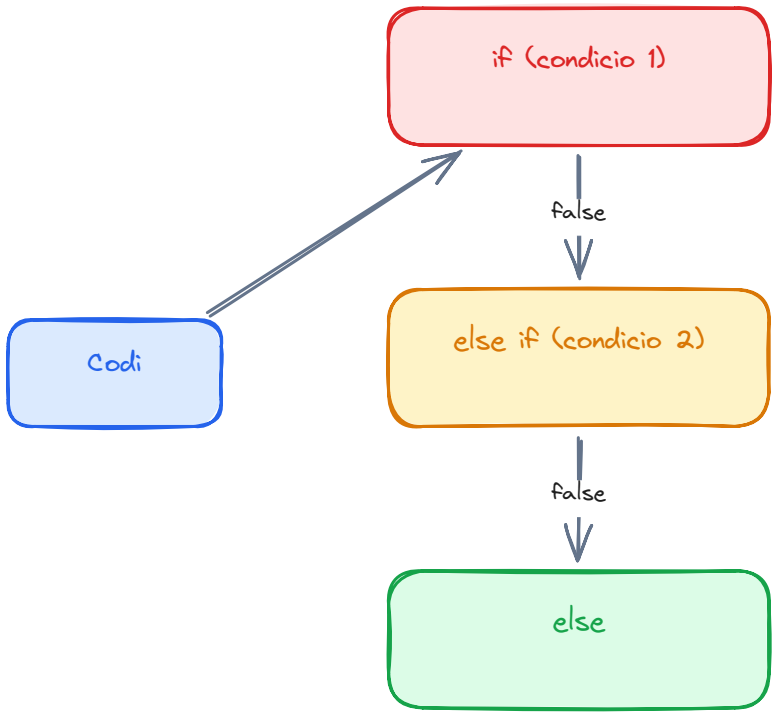
## La idea en una frase

> Fins ara els teus programes anaven de dalt a baix, una línia darrere l'altra. A partir d'ací **decideixen** (if, switch), **repeteixen** (while, for) i **interrompen** (break, continue): el flux deixa de ser una línia recta i es converteix en un mapa.

En este bloc: les condicions, els bucles i les eixides anticipades. Les excepcions esperen al bloc 02.

# El semàfor del flux

## Com s'avalua una cadena d'if




✔ true → s'executa i salta la resta

✔ true → s'avalua si l'anterior va fallar

✔ true → l'últim recorregut (cap dels anteriors)

El primer true guanya: ordena les condicions de la més exigent a la més permissiva.

# if / else if / else: el semàfor del codi

Semafor.java

```
int nota = 7;

if (nota ⩾ 9) {
    System.out.println("Excel·lent");
} else if (nota ⩾ 7) {
    System.out.println("Notable");
} else if (nota ⩾ 5) {
    System.out.println("Aprovat");
} else {
    System.out.println("Suspés");
}
```

ℹ️ Java avalua **de dalt a baix**: tan bon punt una condició done `true`, executa el seu bloc i **salta la resta**. El `else` final atrapa els que no entraren — i és opcional.

# L'ordre mana: del més exigent al més permissiu

### ❌ Ordre invertit

```
if (nota ⩾ 5) {
    resultat = "Aprovat";
} else if (nota ⩾ 7) {
    resultat = "Notable";
}
// amb nota = 8 → "Aprovat"
```

### ✅ Ordre correcte

```
if (nota ⩾ 9) {
    resultat = "Excel·lent";
} else if (nota ⩾ 7) {
    resultat = "Notable";
}
// amb nota = 8 → "Notable"
```

⚠️ **El primer if que es complix guanya**, encara que no siga el que volies. Amb nota = 8, la versió invertida mai no arriba al "Notable": este és el 90% dels bugs de la unitat.

# If anidats i ternari

## If dins d'un if

```java
if (edat ⩾ 18) {
    if (teCarnet) {
        System.out.println("Pots conduir.");
    } else {
        System.out.println("Et falta el carnet.");
    }
}
```

## El ternari: if de butxaca

```java
String msg = (edat ⩾ 18)
    ? "Major d'edat"
    : "Menor d'edat";
```

```
<div class="success" style="margin-top:0.6rem;">
  <strong>Regla d'or:</strong> ternari per a <strong>assignar un valor</strong> en
</div>
<div class="warning" style="margin-top:0.6rem;">
  Més de <strong>3 nivells</strong> d'anidament = Torres Kio: senyal que cal aplana
</div>
```

# Mini-chequeig: if / else

🧠 Sense executar, en 30 segons:

1. Què fa Java si un `if` és false i no hi ha `else`?
2. En quin ordre has d'encadenar les condicions d'un `else if`?
3. `int edat = 16; boolean acompanyat = true;` — què imprimeix: `edat ⩾ 18 ? "Entra" : edat ⩾ 16 && acompanyat ? "Amb acompanyant" : "Fora"`?

🔄 **Respostes:**

- **Continua la següent línia**: l'if s'ignora en silenci
- De la més **exigent** a la més **permissiva**: el primer `true` es queda la decisió
- **"Amb acompanyant"**: 16 no arriba a 18, però sí que compleix la segona condició — el ternari també s'avalua en ordre

# switch: el menú del restaurant

## La idea en una frase

> **switch** és la carta d'un restaurant: mires **el valor d'una variable** i executes el `case` que coincidisca — sense encadenar vint `else if`. Ideal per a **moltes opcions concretes** (dia, talla, menú).

Admet enters, `char`, `enum` i `String` (des de Java 7). Amb rangs o condicions combinades → if/else if.

# switch en acció

MenuDia.java

```
String dia = "dimecres";

switch (dia) {
    case "dilluns":  System.out.println("Llenties"); break;
    case "dimarts":  System.out.println("Paella");   break;
    case "dimecres": System.out.println("Macarrons");break;
    default:         System.out.println("Cap de setmana: no hi ha menú");
}
```

⚠️ **Sense break → fall-through**: el codi es desborda al case següent fins a trobar un break. El clàssic bug del novell.

✅ **default** és el comodí ("cap dels anteriors") i és opcional, com l'else.

# Fall-through: error o superpoder?

## ⚠️ Accidental

```
switch (plat) {
    case 1:
        System.out.println("Amanida");
        // sense break:
        // es cau al següent
    case 2:
        System.out.println("Sopa");
        break;
}
// plat = 1 imprimeix les dues
```

## ✅ Intencional

```
switch (lletra) {
    case 'a': case 'e':
    case 'i': case 'o':
    case 'u':
        System.out.println("Vocal");
        break;
    default:
        System.out.println("Consonant");
}
```

ℹ️ Agrupar casos sense break entre ells és **elegant i compacte**. Oblidar-lo sense voler és un tobogan: **compta els break**, un per cada case no compartit.

# Mini-chequeig: switch

🧠 El switch oblidadís — què imprimeix si `numero = 2` ?

```
switch (numero) {
    case 1:  System.out.println("U");   // sense break
    case 2:  System.out.println("Dos"); // sense break
    case 3:  System.out.println("Tres"); break;
    default: System.out.println("Altre"); break;
}
```

📋

🔁 **Resposta:**

- Imprimeix **Dos** i **Tres**: entra pel case 2, no troba break i **es cau al case 3**, on sí que hi ha break
- El case 1 mai no s'executa perquè numero no val 1 — el fall-through baixa, mai no puja

# Bucles: while i do-while

## La idea en una frase

> Un bucle és una cinta de córrer: executa el mateix bloc una vegada i una altra **mentre la condició siga true**. El `while` comprova **abans** de córrer; el `do-while` corre **almenys una vegada** i comprova després.

Copiar i enganxar cent `println` és pecat: els bucles fan la faena bruta per tu.

# while vs do-while: el duel

## 🏃 while: mira i després corre

```
int intents = 3;
while (intents > 0) {
    System.out.println("Queden " + intents);
    intents = intents - 1;
}
```



```
<div class="info" style="margin-top:0.5rem;">
  Si la condició és false d'entrada: <strong>zero execucions</strong>. Ideal per a llegir fins a un sentinella ("eixir").
</div>
```

## 🏃 do

```
int opcio
do {
    Syste
    Syste
    opcio
} while
```



```
<div clas
  El menú
</div>
```

⚠️ ⚠️ **Bucle infinit:** si dins del bloc res no modifica la condició cap a `false`, el programa no acaba mai. La pregunta obligada: *la condició avança cap a false en algun moment?*

# Mini-chequeig: bucles while

🧠 El comptador parat — quantes voltes fa o es penja?

```
int x = 10;
while (x > 0) {
    System.out.println("Hola");
    x = x + 1;
}
```

📋

🔄 **Resposta:**

- **Bucle infinit**: x comença en 10 i *puja* en comptes de baixar — la condició `x > 0` és true per sempre
- La correcció és `x = x - 1;`. Pista visual: un comptador que puja en un bucle que demana que baixe és fum a l'ordinador

# for i bucles anidats

## La idea en una frase

> **for** és un bucle amb comptador de sèrie: inicialització, condició i actualització en la mateixa línia. Ideal per a **"repeteix N vegades"**. I un bucle dins d'un altre (**anidat**) és una graella: files × columnes.

Si saps quantes vegades repetiràs → `for`. Si no ho saps → `while`.

# L'anatomia del for

TaulaMultiplicar.java

```java
for (int i = 1; i ⩽ 10; i++) {
    System.out.println("7 x " + i + " = " + (7 * i));
}
```

1. `int i = 1` — inicialització: **una sola vegada**, en entrar
2. `i ⩽ 10` — condició: es comprova **abans de cada volta**
3. `i++` — actualització: al final de cada volta

⚠️ La sintaxi és **inici ; condició ; avanç**: tres parts i dos punts i coma, zero comes. I compte amb l'**off-by-one**: `<` fa 9 voltes on `⩽` en fa 10.

ℹ️ while, do-while i for fan el mateix: **tria for quan sàpies les voltes**.

An error occurred on this slide. Check the terminal for more information.

# Mini-chequeig: for i anidats

🧠 Resposta ràpida:

1. Quantes voltes fa `for (int i = 0; i < 3; i++)`?
2. Quantes vegades s'executa la inicialització d'un for?
3. El triangle amb `fila ⩽ 4`: quants asteriscs imprimeix en total?

🔁 **Respostes:**

- **3 voltes**: amb i valent 0, 1 i 2 — quan i arriba a 3 la condició falla
- **Una sola**, en entrar en el bucle
- **10** (1+2+3+4): cada fila imprimeix tants asteriscs com el seu número

# break, continue i etiquetes

## La idea en una frase

> **break** apaga el bucle sencer; **continue** es salta només la volta actual. En bucles anidats afecten el **més intern**, i les **etiquetes** permeten apuntar a un bucle exterior.

Un acaba la festa; l'altre només es perd una cançó.

# break i continue en acció

## 🚪 break: botó de parada

```
for (int i = 1; i ⩽ 10; i++) {
    if (i == 5) {
        break;  // apaga el bucle
    }
    System.out.println(i);
}
// eixida: 1 2 3 4
```

📋

## ⏭️ continue: botó de saltar

```
for (int i = 1; i ⩽ 5; i++) {
    if (i == 3) {
        continue;  // salta la volta
    }
    System.out.println(i);
}
// eixida: 1 2 4 5
```

📋

⚠️ En un **while**, un continue posat **abans d'actualitzar** la variable salta l'actualització… i el bucle no avança: bug infinit assegurat. En un for no passa res: l'actualització viu a la capçalera.

# El bucle psicodèlic

Psicodelic.java

```java
for (int i = 1; i ⩽ 8; i++) {
    if (i % 3 == 0) {
        continue;   // es salta el 3 i el 6
    }
    if (i == 7) {
        break;      // apaga el bucle en el 7
    }
    System.out.println(i);
}
```

ℹ️ **Eixida: 1, 2, 4, 5** — el continue es menja els múltiples de 3, el break talla en el 7 (que tampoc no s'imprimeix) i el 8 mai no s'avalua.

⚠️ Les **etiquetes** ( `exterior:` davant d'un bucle + `break exterior;` ) són legals però poc usades: quasi sempre es pot redissenyar amb una variable booleana. Moderació.

# Mini-chequeig: break i continue

🧠 Últim chequeig del bloc:

1. break i continue en una frase?
2. En bucles anidats, a quin bucle afecten per defecte?
3. Per què és perillós un continue abans de l'actualització en un while?

🔁 **Respostes:**

- **break acaba** el bucle; **continue salta només la volta actual**
- Al **més intern**; per a eixir de dos alhora cal una etiqueta ( `break etiqueta;` )
- L'actualització es salta i el bucle **no avança**: condició true per sempre

# Resum del bloc 01

## Tres frases per quedar-te



### Condicions

if/else if s'avaluen en ordre i guanya el primer true: ordena de més exigent a més permissiva. switch per a valors concrets, amb break per cas.



### Bucles

while comprova abans, do-while almenys una volta, for compta les voltes. La condició ha d'avançar cap a false o tens un bucle infinit.



### Eixides

break apaga el bucle, continue salta la volta; en anidats manen el més intern. Les etiquetes existeixen, però ús-les amb moderació.

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //